Exp-3: Generative Adversarial Network

AIM:

To implement a **Generative Adversarial Network (GAN)** to generate synthetic images using the MNIST dataset.

DATASET:

MNIST Handwritten Digits Dataset

PROCEDURE:

Step 1: Import Libraries

```
import torch
import torch.nn as nn
import torchvision import torchvision.transforms as transforms
import matplotlib.pyplot as plt
```

Step 2: Load Dataset

```
transform = transforms.Compose([ transforms.ToTensor(),
transforms.Normalize((0.5,), (0.5,))]) dataset =
torchvision.datasets.MNIST(root='./data', download=True, transform=transform)
dataloader = torch.utils.data.DataLoader(dataset, batch_size=64, shuffle=True)
```

Step 3: Define Generator

```
class
Generator(nn.Module):
def init(self):
super(Generator,
self).init()
```

self.model = nn.Sequential(nn.Linear(100, 256), nn.ReLU(), nn.Linear(256, 512), nn.ReLU(), nn.Linear(512, 784), nn.Tanh()) def forward(self, x):

return self.model(x)

Step 4: Define Discriminator

class Discriminator(nn.Module):

def **init**(self):

super(Discriminator, self).**init**()

self.model = nn.Sequential(nn.Linear(784, 512), nn.LeakyReLU(0.2), nn.Linear(512, 256), nn.LeakyReLU(0.2), nn.Linear(256, 1), nn.Sigmoid()) def forward(self, x):

return self.model(x)

Step 5: Initialize Models

generator = Generator()

discriminator = Discriminator()

Step 6: Loss and Optimizer criterion = nn.BCELoss() g_optimizer = torch.optim.Adam(generator.parameters(), lr=0.0002)

d_optimizer = torch.optim.Adam(discriminator.parameters(), lr=0.0002)

Step 7: Training GAN

epochs = 10 for epoch in range(epochs): for images, _ in dataloader:

images = images.view(-1, 784)

real_labels = torch.ones(images.size(0), 1)

fake_labels = torch.zeros(images.size(0), 1)

outputs = discriminator(images)

```
d_loss_real =criterion(outputs, real_labels)
noise = torch.randn(images.size(0), 100)
fake_images = generator(noise)
outputs = discriminator(fake_images.detach())
d_loss_fake = criterion(outputs, fake_labels)
d_loss = d_loss_real + d_loss_fake d_optimizer.zero_grad() d_loss.backward()
d_optimizer.step() outputs = discriminator(fake_images) g_loss =
criterion(outputs, real_labels) g_optimizer.zero_grad() g_loss.backward()
g_optimizer.step() print(f"Epoch [{epoch+1}/{epochs}] | D Loss: {d_loss.item()} |
G Loss: {g_loss.item()}")
```

Step 8: Generate Images

```
noise = torch.randn(16, 100)
generated_images = generator(noise).detach()
generated_images = generated_images.view(-1, 28,
28) fig, axes = plt.subplots(4, 4, figsize=(5,5)) for i,
ax in enumerate(axes.flat):
ax.imshow(generated_images[i],
cmap='gray') ax.axis('off') plt.show()
```

OUTPUT:

```
Using: cpu
Epoch [1/5] | D Loss: 0.9090 | G Loss: 1.8620
Epoch [2/5] | D Loss: 1.0690 | G Loss: 2.4561
Epoch [3/5] | D Loss: 1.1777 | G Loss: 0.6676
Epoch [4/5] | D Loss: 1.0934 | G Loss: 0.8375
Epoch [5/5] | D Loss: 1.0666 | G Loss: 1.2042
```

RESULT:

The GAN successfully learns to generate handwritten digit images similar to MNIST. Initially, outputs are noisy, but improve with training as the generator and discriminator compete.